Practical 5: K-Nearest Neighbors on the Diabetes Dataset

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
recall_score, classification_report
```

In [2]:
```
df = pd.read_csv("diabetes.csv")
df.head()
```

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

In [3]:
```
df.shape
```

Out[3]:
```
(768, 9)
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Pregnancies    768 non-null    int64  
 1   Glucose        768 non-null    int64  
 2   BloodPressure  768 non-null    int64  
 3   SkinThickness  768 non-null    int64  
 4   Insulin        768 non-null    int64  
 5   BMI            768 non-null    float64
 6   Pedigree       768 non-null    float64
 7   Age            768 non-null    int64  
 8   Outcome        768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [5]:
```
df.describe()
```

Out[5]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
Pedigree         0
Age              0
Outcome          0
dtype: int64
```

In [7]:
```
zero_columns = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
(df[zero_columns] == 0).sum()
```

Out[7]:
```
Glucose            5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI               11
dtype: int64
```

In [8]:
```
for col in zero_columns:
    df[col] = df[col].replace(0, np.nan)
    df[col] = df[col].fillna(df[col].median())
(df[zero_columns] == 0).sum()
```

Out[8]:
```
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
dtype: int64
```

In [9]:
```
df["Outcome"].value_counts()
```

Out[9]:
```
Outcome
0    500
1    268
Name: count, dtype: int64
```

In [10]:
```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

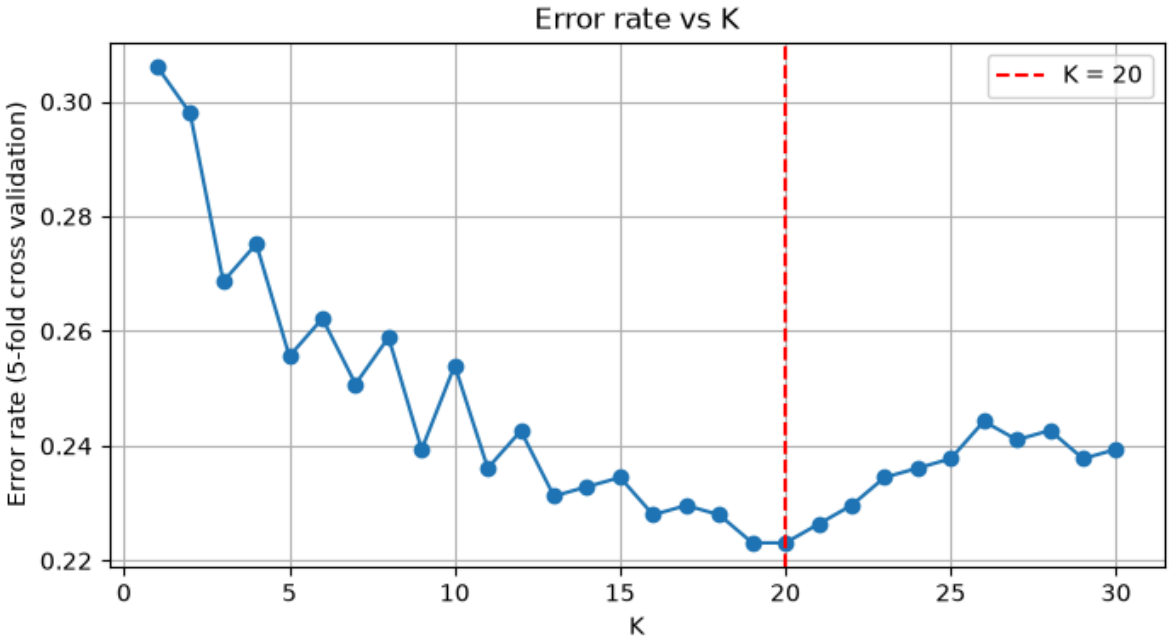
```
Training set: (614, 8)
Test set: (154, 8)
```

In [11]:
```
k_values = range(1, 31)
cv_error = []
for k in k_values:
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train, cv=5,
scoring="accuracy")
    cv_error.append(1 - scores.mean())

best_k = k_values[int(np.argmin(cv_error))]
print("Best value of K:", best_k)

plt.figure(figsize=(8, 4))
plt.plot(k_values, cv_error, marker="o")
plt.axvline(best_k, color="red", linestyle="--", label=f"K = {best_k}")
plt.xlabel("K")
plt.ylabel("Error rate (5-fold cross validation)")
plt.title("Error rate vs K")
plt.legend()
plt.grid(True)
plt.show()
```

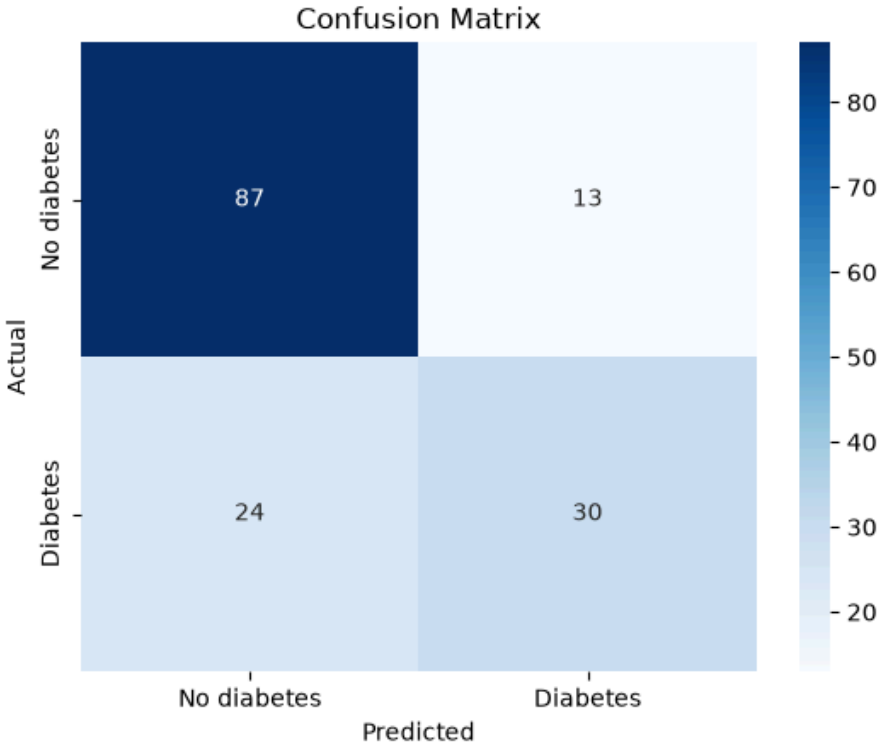
```
Best value of K: 20
```



In [12]:
```
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

In [13]:
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No diabetes", "Diabetes"], yticklabels=["No diabetes",
"Diabetes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
Confusion Matrix:
[[87 13]
 [24 30]]
```



In [14]:
```
tn, fp, fn, tp = cm.ravel()
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("True Negatives :", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives :", tp)
print()
print("Accuracy  :", round(accuracy, 4))
print("Error rate:", round(error_rate, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
```

```
True Negatives : 87
False Positives: 13
False Negatives: 24
True Positives : 30

Accuracy  : 0.7597
Error rate: 0.2403
Precision : 0.6977
Recall    : 0.5556
```

In [15]:
```
print(classification_report(y_test, y_pred, target_names=["No diabetes", "Diabetes"]))
```

```
              precision    recall  f1-score   support

 No diabetes       0.78      0.87      0.82       100
    Diabetes       0.70      0.56      0.62        54

    accuracy                           0.76       154
   macro avg       0.74      0.71      0.72       154
weighted avg       0.75      0.76      0.75       154
```